AI Daily Sales Plan Architecture Review & Strengthening Report

System: Field Sales Management (FSM) Application
Target Codebase: pharmaHRMS_backend (FastAPI, SQLAlchemy, PostgreSQL/Supabase) & Pharma_HRMS (React 18, Vite, Tailwind CSS)
Baseline Architecture Document: AI_Daily_Sales_Plan_Architecture_Design.md
Review Status: Codebase & Architecture Audit Completed
Implementation Status: HOLD — Pending Architectural Corrections & Stakeholder Sign-Off

---

1. Executive Summary

This architecture review provides an exhaustive evaluation of the proposed **AI Daily Sales Plan feature** against the 18 design gaps identified during peer review. The review cross-references the initial architecture documented in AI_Daily_Sales_Plan_Architecture_Design.md with the active repository implementation across database schemas, FastAPI backend routers, and React frontend services.

Key Audit Findings

1. **The Feature Currently Exists Solely as a Design Document:**
   No database tables, migrations, SQLAlchemy models, FastAPI plan routers, or frontend plan services exist yet for `leads`, `follow_ups`, `monthly_targets`, `daily_sales_plans`, or `daily_sales_plan_items`. The repository currently contains manual visit scheduling on pharma_hrms_field_visits and general tasks on pharma_hrms_tasks.
2. **Missing Multi-Obligation Junction Model:**
   The initial design assumed a 1:1 relationship between a planned physical stop and a single lead record. In field reality, a single clinic stop often represents multiple obligations (e.g., an overdue follow-up callback, a routine promotional visit, and a manager-mandated document delivery). Without an explicit junction model (`daily_sales_plan_item_obligations`), separate source obligations cannot be consolidated, tracked, or fulfilled independently.
3. **Flawed Authorization & RLS Assumptions:**
   The initial architecture assumed Supabase Row Level Security (RLS) would enforce `auth.uid() = assigned_to`. In the actual codebase, client authentication uses custom FastAPI JWTs (app/routers/auth.py), the Supabase client operates with an anonymous key without persistent sessions (src/core/supabase.ts), and active PostgreSQL RLS policies are completely permissive

(`CREATE POLICY "%s_all" ON "%I" FOR ALL USING (true);`). All authorization and hierarchy checks must be strictly enforced server-side within FastAPI.

4. **Static Projection vs. Dynamic Reconciliation:**
   The original design treated a daily sales plan as an immutable, static snapshot. It lacked mechanisms for out-of-band completion reconciliation, lead reassignment invalidation, mandatory stop feasibility verification, and controlled midday replanning.

Architecture Verdict

The architecture is NOT YET ready for implementation. While the foundational mathematical components (MCDA priority scoring, OSRM road matrices, Nearest Neighbor, and 2-Opt TSPTW) are sound and must be preserved, the data contracts, relational schemas, and lifecycle reconciliation mechanisms require structural strengthening before coding begins.

---

2. Existing Architecture Verified from the Repository

The codebase was inspected to establish the baseline implementation before evaluating the 18 peer-review findings:

```
d:/FSM/
├── AI_Daily_Sales_Plan_Architecture_Design.md  # 81 KB Architecture Specification Document
├── Pharma_HRMS/                                # React 18 + Vite + Tailwind CSS Frontend
│   └── src/
│       ├── components/
│       │   ├── DoctorVisitPlanner.tsx          # Manual visit calendar & assignment UI
│       │   ├── CheckInModule.tsx               # GPS attendance & check-in module
│       │   └── fieldops/
│       │       ├── ModalPreviewMap.tsx         # Leaflet route preview
│       │       └── CallPhotoCaptureView.tsx    # Camera proof capture
│       ├── lib/
│       │   ├── osrm.ts                         # Public OSRM router with 6s timeout & Haversine fallback
│       │   ├── geofence.ts                     # Haversine distance & geofence validation
│       │   ├── fieldOpsConfig.ts               # Coordinates & endpoints configuration
```

```
|       |   └── services/
|       |       ├── field-visit-service.ts      # Visit state machine
(ASSIGNED -> EN_ROUTE -> ARRIVED -> COMPLETED)
|       |       ├── task-service.ts             # Task CRUD and approval
workflows
|       |       └── attendance-service.ts       # Attendance and punch
records
|       └── core/
|           └── supabase.ts                     # Supabase JS client
(initialized with anon key, no auth persistence)
└── pharmaHRMS_backend/                         # FastAPI + SQLAlchemy
Backend
    ├── app/
    |   ├── main.py                             # API Gateway mounting
auth, employees, field, tasks, attendance
    |   ├── dependencies/auth.py                # JWT decode,
get_current_user, role & hierarchy evaluation
    |   ├── models/
    |   |   ├── field.py                        # FieldSession,
FieldVisit, LocationPin, DoctorSubmission
    |   |   ├── task.py                         # Task and Notification
models
    |   |   └── employee.py                     # Employee model (35
personnel seeded)
    |   └── routers/
    |       ├── field.py                        # Session start/end, visit
CRUD, visit complete, location pins
    |       └── tasks.py                        # Task CRUD, status
updates, manager approvals
    └── database/
        ├── 00_pharma_hrms_full_migration.sql   # Core schema, 35 employee
seed roster, permissive RLS
        └── 08_client_handover_reset.sql        # pharma_hrms_tasks
definition & cleanup
```

Verified Codebase Realities

- **Commercial Records:** The `leads` table does not exist. Currently, doctors and clinics are stored as unindexed text columns (`doctor_name` , `clinic_name` , `area`) inside pharma_hrms_field_visits or as unverified prospect submissions in pharma_hrms_doctor_list_submissions.

- **Field Visit Execution:** The field-visit state machine is fully implemented in field-visit-service.ts and app/routers/field.py. It tracks `ASSIGNED` $\to$ `EN_ROUTE` $\to$ `ARRIVED` $\to$ `COMPLETED`, measures arrival distance against `allowed_radius_meters`, logs `arrival_distance_m`, and flags `location_exception`.
- **Manager Tasks:** The table pharma_hrms_tasks exists and supports `priority` (`low`, `medium`, `high`, `urgent`), `status` (`in_progress`, `completed`, `incomplete`), `due_date`, `assigned_to`, and `approval_status`.
- **Routing Engine:** Pharma_HRMS/src/lib/osrm.ts implements `fetchRoute()` querying `https://router.project-osrm.org` with a 6-second abort controller. If the network drops or times out, it falls back to Haversine distance with a $1.2\times$ road-winding factor and an assumed 30 km/h speed.
- **Authentication & Authorization Boundary:** Client authentication uses custom JWTs via `/api/auth/login` verified by app/dependencies/auth.py. The database RLS policies in 00_pharma_hrms_full_migration.sql execute `CREATE POLICY "%s_all" ON "%I" FOR ALL USING (true) WITH CHECK (true);`. Therefore, **Supabase PostgREST does not enforce access control; all authorization checks must reside server-side in FastAPI.**

---

3. Assessment of All 18 Peer-Review Issues

#	Peer Review Issue	Assessment Status	Primary Affected Layer	Root Cause Summary
1	Duplicate daily plan generation	**Partially addressed**	Backend / Database	Unique constraint exists in design text, but lacks atomic distributed locks, idempotency keys, and revision models.
2	Missing links to source records	**Missing**	Database / Backend	Plan items only link to `lead_id`. Foreign keys to `follow_ups`, `tasks`, and `field_visits` are missing.
3	Duplicate visits from multiple sources	**Missing**	Architecture / Logic	No candidate deduplication or junction architecture to represent multiple obligations at one physical stop.
4	Lead or task reassignment after generation	**Missing**	Backend / Workflow	No execution-time ownership revalidation; previous assignee retains access to execute out-of-date plan stops.

#	Peer Review Issue	Assessment Status	Primary Affected Layer	Root Cause Summary
5	External completion not reflected in the plan	**Missing**	Backend / Data Sync	Plan items do not reconcile with authoritative source states; tasks closed externally remain pending in the plan.
6	Historical plan integrity	**Partially addressed**	Database / Analytics	In-place updates overwrite original AI recommendations, destroying plan vs. actual compliance audit trails.
7	Backlog carry-forward	**Partially addressed**	Backend / Scoring	Overdue follow-ups are boosted, but uncompleted non-follow-up obligations and task carry-forward rules are undefined.
8	Incorrect backlog prioritization	**Partially addressed**	Algorithm / Business Rule	Overdue score normalization contains ad-hoc boosts; risks chronic backlog starving newly confirmed appointments.
9	Closed or cancelled source after plan creation	**Missing**	Backend / Execution	Candidate filtering only runs during generation; no pre-execution guard checks whether a lead was lost or cancelled.
10	Employee leave, exit, or unavailability	**Missing**	Backend / Integration	No integration with `pharma_hrms_leave_requests`; unexecuted plans of unavailable reps are not safely abandoned or reassigned.
11	Undefined meeting-time estimates	**Partially addressed**	Algorithm / Config	Hardcodes a single 45-minute duration; lacks activity categorization, confirmed appointment windows, and lunch breaks.
12	Infeasible mandatory assignments	**Missing**	Algorithm / Workflow	Mandatory stops unconditionally bypass budget checks; no overlap detection or manager conflict escalation.
13	Midday changes and controlled	**Partially addressed**	Backend / API	Text describes locking completed stops, but lacks API contracts, in-

#	Peer Review Issue	Assessment Status	Primary Affected Layer	Root Cause Summary
	regeneration			progress stop protection, delay cascade, and revision tracking.
14	Route service or optimization failure	**Partially addressed**	Backend / Routing	Frontend has OSRM fallback, but backend optimizer lacks matrix failure handling, disconnected node recovery, and two-phase commit.
15	Standalone follow-ups omitted	**Partially addressed**	Architecture / Data Model	Architecture forces all follow-ups to have a `lead_id`, but organizational tasks without leads are ignored by candidate ingestion.
16	Non-visit actions handled as route stops	**Missing**	Architecture / Algorithm	Calls, quotations, and administrative tasks are routed as geographic stops or omitted from the day's time budget.
17	Plan status confused with execution status	**Partially addressed**	Architecture / State Machine	Physical checkout blindly marks work complete without structured business outcome verification or lead-stage validation.
18	Authorization and source ownership drift	**Missing**	Security / Backend	Database RLS is completely open (`USING (true)`); Supabase `auth.uid()` is null; server-side execution checks are absent.

---

4. Detailed Explanation and Solution for Each Issue

Issue 1: Duplicate Daily Plan Generation

- **Current implementation status:** Partially addressed.
- **What is missing:** The architecture document specifies `UNIQUE(employee_id, plan_date)` on `daily_sales_plans` (Architecture Doc: L698), but lacks concurrency protection against simultaneous clicks, idempotency key support, and revision handling when an approved plan requires regeneration.
- **Example scenario:** Representative Rajesh opens the app in a poor network zone. He taps "Generate Plan" twice in rapid succession. Two concurrent worker threads receive the request.

Thread A checks if a draft exists (finds none) and begins OSRM calculations; Thread B does the same. Both threads calculate itineraries and attempt simultaneous inserts, leading to duplicate stop items or a race condition that corrupts stop sequences.

- **Technical impact:** Violates database consistency, creates orphaned `daily_sales_plan_items`, and triggers unhandled HTTP 500 errors on the client.
- **Recommended solution:**
  1. Add an `idempotency_key` (UUID) to the generation payload.
  2. Implement PostgreSQL transactional advisory locking or `SELECT ... FOR UPDATE` on employee planning records: `pg_advisory_xact_lock(hashtext(employee_id || plan_date::text))`.
  3. If a plan exists in `DRAFT` status, update it within the lock. If `APPROVED` or `IN_PROGRESS`, reject repeated generation unless an explicit `force_regenerate=true` flag is supplied by an authorized manager, which archives the old plan with `status = 'SUPERSEDED'`.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/routers/daily_plan.py`, table `daily_sales_plans`.
- **Dependencies:** Database migration for `daily_sales_plans`.
- **Acceptance criteria:** Concurrent POST requests with the same employee ID and date return the identical plan ID without creating duplicate stop items or throwing constraint violations.

Issue 2: Missing Links to Source Records

- **Current implementation status:** Missing.
- **What is missing:** In Architecture Doc: L729, `daily_sales_plan_items` only has a foreign key to `lead_id`. It has no foreign keys to `follow_ups(id)`, `pharma_hrms_tasks(id)`, or `pharma_hrms_field_visits(id)`. Source reasons are stored as unlinked text strings in a JSONB array.
- **Example scenario:** Rajesh visits Apollo Clinic to fulfill high-priority follow-up `#FU-902` and manager directive `#TASK-412`. Upon completion, the system logs `COMPLETED` on the plan item, but because there are no foreign keys to `#FU-902` or `#TASK-412`, neither the follow-up nor the manager task is marked complete in their respective modules.
- **Technical impact:** Broken referential integrity, inability to trace why a stop was scheduled, and orphaned open commitments in the follow-up and task systems.
- **Recommended solution:**
  1. Create a dedicated junction table: `daily_sales_plan_item_obligations`.
  2. Each record maps `plan_item_id` to `source_type` (`LEAD_CONVERSION`, `FOLLOW_UP`, `MANAGER_TASK`), `source_id` (UUID), `obligation_title`, and

individual `fulfillment_status` (`PENDING`, `FULFILLED`, `UNFULFILLED`, `RESCHEDULED`).

3. Add nullable `field_visit_id` (UUID referencing `pharma_hrms_field_visits`) to `daily_sales_plan_items` to preserve the link to the actual visit execution record.

- **Files/tables/APIs affected:** New table `daily_sales_plan_item_obligations`, table `daily_sales_plan_items`.
- **Dependencies:** Table creation migrations.
- **Acceptance criteria:** Every stop item explicitly displays its linked follow-up and task IDs; checking off a stop updates all associated source obligations through stable foreign keys.

Issue 3: Duplicate Visits from Multiple Sources

- **Current implementation status:** Missing.
- **What is missing:** No consolidation logic exists when a single doctor/clinic is targeted by a lead record, an overdue callback, and a manager task on the same day.
- **Example scenario:** Manager Naidu creates an urgent task: *"Deliver clinical cardiac trial brochure to Dr. Patnaik."* Simultaneously, the system has an existing follow-up due for Dr. Patnaik: *"Collect signed proposal."* The candidate generator evaluates both and adds Dr. Patnaik as Stop #1 (for the task) and Stop #4 (for the follow-up), forcing Rajesh to travel to the same clinic twice in one day.
- **Technical impact:** Severe route inefficiency, wasted travel time and fuel expenses, and customer frustration from multiple visits on the same date.
- **Recommended solution:**
  1. During candidate ingestion, group all obligations by `lead_id` (or geographic coordinate cluster within 50 meters).
  2. Consolidate compatible obligations into a **single physical stop**.
  3. Attach all constituent obligations to that stop via `daily_sales_plan_item_obligations`.
  4. Aggregate the planned duration: base visit time + incremental time per additional obligation (e.g., $30\text{ min} + 15\text{ min} = 45\text{ min}$).
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/plan_optimizer.py`.
- **Dependencies:** Issue 2 junction table.
- **Acceptance criteria:** If multiple obligations exist for the same lead, the generator creates exactly one route stop containing all obligations in its checklist.

Issue 4: Lead or Task Reassignment After Generation

- **Current implementation status:** Missing.

- **What is missing:** No authorization re-validation or plan invalidation occurs if a manager reassigns a lead to another representative after the morning plan has been generated and approved.
- **Example scenario:** Rajesh's plan is approved at 08:30 AM with Lead A at 11:00 AM. At 09:30 AM, Regional Manager Naidu reassigns Lead A to Representative Shiva due to territory redistricting. Rajesh arrives at Lead A at 11:00 AM and executes GPS check-in. The system permits Rajesh to check in and log proprietary customer discussion notes for an account he no longer owns.
- **Technical impact:** Commercial data leakage, territory boundary violations, and corrupted commission/target attribution.
- **Recommended solution:**
  1. Add an execution-time check in `/api/field/visits` and `/api/v1/plans/items/{id}/start`: re-verify that `lead.assigned_to == current_user.id` (or current user is delegated).
  2. If reassigned, reject execution with HTTP 409 Conflict: *"Lead has been reassigned to another representative."*
  3. Automatically transition the plan item to `REASSIGNED` and recalculate downstream ETAs.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/routers/field.py`, `app/services/plan_reconciliation.py`.
- **Dependencies:** Core `leads` table schema.
- **Acceptance criteria:** Attempting to start or check into a stop for a lead that was reassigned after plan creation is blocked with a clear explanatory error.

Issue 5: External Completion Not Reflected in the Plan

- **Current implementation status:** Missing.
- **What is missing:** If a follow-up or manager task linked to a plan item is completed outside the plan UI (e.g., via the Tasks screen app/routers/tasks.py or a direct phone call), the saved plan item remains `PENDING`.
- **Example scenario:** Rajesh finishes Stop #1, then calls Doctor B from the car to answer an urgent query. Doctor B is satisfied, so Rajesh opens the Follow-ups tab and marks `#FU-881` as `COMPLETED`. However, his active daily plan still lists Doctor B as Stop #3 (`PENDING`). Rajesh's route map still directs him to drive to Doctor B's clinic.
- **Technical impact:** Out-of-sync plan status, redundant driving, and inaccurate end-of-day plan adherence metrics.
- **Recommended solution:**
  1. Implement an active reconciliation pass in `GET /api/v1/plans/daily`: before returning the plan payload, query the authoritative status of all linked source records in `daily_sales_plan_item_obligations`.
  2. If all obligations for a stop are resolved externally, transition the plan item status to `EXTERNALLY_RESOLVED` and provide the representative with a 1-tap option: *"Stop #3 was*

resolved externally. Remove from route and recalculate ETAs?"

- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/routers/daily_plan.py`, `Pharma_HRMS/src/components/DoctorVisitPlanner.tsx`.
- **Dependencies:** Issue 2 junction table.
- **Acceptance criteria:** Marking a task or follow-up complete in its native module updates the plan item status upon the next plan fetch or mobile sync.

Issue 6: Historical Plan Integrity

- **Current implementation status:** Partially addressed.
- **What is missing:** In Architecture Doc: L1172, plan updates and rep customizations mutate `daily_sales_plans` and `daily_sales_plan_items` in place. Original AI recommendations, initial scores, initial stop sequences, and approved baselines are completely overwritten.
- **Example scenario:** The AI engine recommends 6 stops. Rajesh drags Stop #6 to Stop #1 and deletes Stop #2 because it is far away. In the evening, Analytics evaluates plan adherence. Because the original recommendation was overwritten in place, management cannot evaluate whether Rajesh followed the AI recommendation or whether his manual override reduced revenue yield.
- **Technical impact:** Destruction of historical audit trails, inability to compute plan adherence analytics, and lack of accountability for manual deviations.
- **Recommended solution:**
  1. Introduce plan revision versioning: `daily_sales_plans` has columns `version` (INT, default 1) and `revision_type` (`AI_INITIAL`, `REP_CUSTOMIZED`, `MANAGER_MODIFIED`, `MIDDAY_REPLAN`).
  2. When a representative customizes a plan or a midday replan occurs, archive the previous state into `daily_sales_plan_revisions` (or increment version with an immutable snapshot) before saving the active version.
  3. Ensure stop items record `initial_sequence_order`, `customized_sequence_order`, and `actual_execution_order`.
- **Files/tables/APIs affected:** Table `daily_sales_plans`, new table `daily_sales_plan_revisions`.
- **Dependencies:** Database migration.
- **Acceptance criteria:** The system preserves both the original AI recommendation and all subsequent approved revisions, allowing side-by-side comparison in executive analytics.

Issue 7: Backlog Carry-Forward

- **Current implementation status:** Partially addressed.
- **What is missing:** The scoring engine applies an overdue boost to follow-ups (Architecture Doc: L974), but there is no architectural policy for uncompleted non-follow-up visits or open manager tasks. The system lacks a clean end-of-day rollover mechanism that avoids cloning records.
- **Example scenario:** Rajesh is assigned 6 stops today. Due to severe rain, he only completes 3 stops. Stop #4 was an AI-recommended lead visit with no scheduled follow-up. Stop #5 was a manager task. At midnight, neither stop was executed. If the system creates new tasks for these tomorrow, source records are duplicated. If the system ignores them, unvisited commitments disappear.
- **Technical impact:** Orphaned commitments, uncontrolled task duplication, and loss of territory accountability.
- **Recommended solution:**
  1. **Strict Non-Cloning Rule:** Never clone or duplicate source leads, tasks, or follow-ups upon plan expiration.
  2. At day end (23:59:59) or upon next morning plan generation, mark uncompleted plan items in yesterday's plan as `UNVISITED` with a carry-forward flag.
  3. In the candidate selection query for tomorrow's plan, pull all open obligations directly from their authoritative tables:
     - Open tasks: `pharma_hrms_tasks WHERE assigned_to = emp_id AND status = 'in_progress'`
     - Open follow-ups: `follow_ups WHERE employee_id = emp_id AND status = 'PENDING' AND scheduled_date <= tomorrow`
     - Active leads: `leads WHERE assigned_to = emp_id AND stage NOT IN ('WON', 'LOST', 'DISQUALIFIED')`
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/plan_optimizer.py`, `app/jobs/daily_rollover.py`.
- **Dependencies:** Core schema models.
- **Acceptance criteria:** Unfinished stops from yesterday appear as candidates for today's plan using their original IDs, without generating duplicate task or follow-up rows.

Issue 8: Incorrect Backlog Prioritization

- **Current implementation status:** Partially addressed.
- **What is missing:** The MCDA formula includes U_i (Urgency) and D_i (Recency), but the scoring normalization contains arbitrary bonuses (e.g., the undocumented $+21.75$ "overdue bonus" in Architecture Doc: L1010). Chronic overdue backlog can monopolize the plan and starve newly confirmed, high-value appointments.
- **Example scenario:** Rajesh has 10 follow-ups that are 20 days overdue for low-value replenishment queries (5,000 INR each). Today, a major hospital director (deal value 400,000 INR) has an

appointment scheduled for 10:00 AM. Because the 10 old follow-ups all max out the overdue urgency factor ($U_i = 100$), they crowd out the hospital appointment from the top-6 candidate list.

- **Technical impact:** High-value deals stall, time-sensitive appointments are dropped from recommendations, and field representatives disregard the AI plan because it recommends stale backlog.
- **Recommended solution:**
  1. Restructure the Urgency factor U_i into four distinct non-linear tiers:
     - **Confirmed Appointment Due Today:** $U_i = 100$ (must be scheduled at its exact appointment window).
     - **Grace-Period Overdue (1–3 days overdue):** $U_i = 85$.
     - **Follow-up Due Today (flexible time):** $U_i = 70$.
     - **Chronic Backlog (>7 days overdue):** Cap at $U_i = 40$, and trigger an alert in `revenue_leakage_alerts` for manager intervention / rescheduling rather than forcing it into today's driving route.
  2. Strictly lock stops with confirmed appointment times into the candidate pool before filling remaining capacity with backlog and high-value prospecting leads.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/lead_scorer.py`.
- **Dependencies:** Multi-factor scoring configuration.
- **Acceptance criteria:** A newly confirmed 400,000 INR appointment scheduled for today is prioritized over 14-day-old low-value backlog items.

Issue 9: Closed or Cancelled Source After Plan Creation

- **Current implementation status:** Missing.
- **What is missing:** Candidate filtering only runs when the plan is generated (Architecture Doc: L250). If a lead is marked `LOST` or a follow-up is cancelled midway through the workday, the stop remains active in the saved plan.
- **Example scenario:** At 11:30 AM, Doctor C's clinic calls the head office and cancels their contract negotiation, marking the lead as `LOST` in the system. Rajesh is currently driving toward Doctor C as Stop #4. His mobile itinerary still shows Doctor C as a valid stop, and he conducts an embarrassing, unannounced visit.
- **Technical impact:** Unprofessional field interactions, wasted representative time, and potential check-in data recorded against a closed commercial entity.
- **Recommended solution:**
  1. Add an active source status validation hook inside the visit execution flow: before `field-visit-service.ts` transitions a stop to `EN_ROUTE` or `ARRIVED`, verify via FastAPI that the underlying lead stage is not `WON`, `LOST`, or `DISQUALIFIED`.
  2. If the lead is closed/cancelled, FastAPI returns HTTP 410 Gone: *"Lead has been marked LOST/CANCELLED."*

3. The stop item transitions to `CANCELLED_AT_SOURCE` and is visually greyed out on the map, prompting route recalculation.

- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/routers/field.py`, `Pharma_HRMS/src/lib/services/field-visit-service.ts`.
- **Dependencies:** Leads table lifecycle.
- **Acceptance criteria:** When a lead is marked `LOST`, subsequent attempts to start route or check into that lead from an existing plan are blocked and the route updates.

Issue 10: Employee Leave, Exit, or Unavailability

- **Current implementation status:** Missing.
- **What is missing:** The architecture does not check employee leave records (pharma_hrms_leave_requests) during plan generation. If an employee is on approved leave, plans can still be generated and approved for them. If an employee takes emergency leave midday, there is no workflow to reassign their outstanding stops.
- **Example scenario:** Business Executive Naveen is granted Casual Leave for October 22. The batch planner runs at 07:00 AM and generates an 8-stop route for Naveen. Naveen does not work. All 8 stops remain in `PENDING` status, creating false delinquency alerts in the evening analytics and leaving urgent customer commitments unserved.
- **Technical impact:** Erroneous scheduling for absent staff, neglected customer accounts, and distorted attendance and visit completion KPIs.
- **Recommended solution:**
  1. **Leave Pre-Condition:** At plan generation, query `pharma_hrms_leave_requests` for the target employee and date where `status = 'approved'`. If approved leave exists, abort plan generation with HTTP 422: *"Employee is on approved leave."*
  2. **Emergency Leave / Midday Absence Workflow:** If an employee marks emergency leave or checks out early from attendance, their active plan transitions to `SUSPENDED` / `ABANDONED`.
  3. Provide an RSM / Manager action: **"Reassign Remaining Stops"**, allowing the manager to transfer the uncompleted obligations directly into a peer representative's plan for that afternoon.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/plan_optimizer.py`, `pharmaHRMS_backend/app/routers/daily_plan.py`.
- **Dependencies:** `pharma_hrms_leave_requests` table.
- **Acceptance criteria:** Plan generation is blocked for employees on approved leave; managers can reassign unexecuted stops of an unavailable employee to another representative with a single action.

Issue 11: Undefined Meeting-Time Estimates

- **Current implementation status:** Partially addressed.
- **What is missing:** The algorithm hardcodes a single 45-minute service duration for all stops (Architecture Doc: L1046), ignores visit categories, ignores confirmed appointment windows, and provides no concrete lunch or administrative break model.
- **Example scenario:** Rajesh has 4 quick retail pharmacy replenishment checks (which take 10 minutes each) and 2 hospital committee meetings (which take 60 minutes each). The algorithm assigns 45 minutes to every stop. It overestimates the pharmacy time by 140 minutes and underestimates the hospital meetings by 30 minutes, producing completely inaccurate arrival ETAs that cause Rajesh to miss the doctors' operating hours.
- **Technical impact:** Infeasible arrival ETAs, violated doctor time windows, and rep abandonment of system-generated schedules.
- **Recommended solution:**
  1. Define a configurable duration matrix based on `category` and `lead_stage`:
     - Routine Doctor Detailing: **20 minutes**
     - Key Opinion Leader (KOL) / Specialist: **35 minutes**
     - Hospital Committee / Proposal Presentation: **60 minutes**
     - Retail Chemist / Pharmacy Stock Check: **15 minutes**
     - Mandatory Midday Lunch Break: **45 minutes** (locked between 13:00 and 14:15)
  2. Clearly distinguish between:
     - `estimated_duration_minutes` (from category configuration)
     - `confirmed_appointment_time` (fixed arrival constraint, e.g. exactly 11:00 AM)
     - `actual_duration_minutes` (calculated from GPS check-in to checkout)
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/route_optimizer.py`, `Pharma_HRMS/src/lib/constants/shifts.ts`.
- **Dependencies:** Visit category schemas.
- **Acceptance criteria:** Itinerary durations and departure times reflect the specific category of each stop rather than a uniform 45-minute constant, and include a protected 45-minute lunch break.

Issue 12: Infeasible Mandatory Assignments

- **Current implementation status:** Missing.
- **What is missing:** In Architecture Doc: L990, stops marked `MANAGER_MANDATORY` unconditionally bypass scoring and are locked into the itinerary. The algorithm only sheds *non-mandatory* stops when the 8.5-hour budget is exceeded (Architecture Doc: L1052). If mandatory stops alone exceed the budget or have conflicting time windows, the system returns an impossible schedule.

- **Example scenario:** Manager Naidu assigns 4 mandatory visits in different towns across the Visakhapatnam rural zone. Total driving time between these locations is 6.5 hours. Total meeting durations equal 3.5 hours. Combined with lunch (0.75h), the mandatory work requires 10.75 hours, exceeding the maximum 8.5-hour working day. Furthermore, Doctor A is only available 10:00–11:00 AM in the North, and Doctor B is only available 10:30–11:30 AM in the South (40 km away).
- **Technical impact:** The TSPTW solver returns an infeasible route where stops violate operating windows, forcing the representative into extreme overtime or guaranteed missed calls.
- **Recommended solution:**
    1. **Mandatory Feasibility Validator:** Before running 2-Opt, validate mandatory stops against total shift duration and pairwise time-window reachability: $$\text{Arrival}_B \ge \text{Departure}A + T{\text{road}}(A, B)$$
    2. If mandatory stops cannot be feasibly scheduled within the 8.5-hour shift budget without window violations, the engine must **not** return a falsely feasible plan.
    3. Instead, return HTTP 422 with a structured conflict payload:
    ```
    {"error": "MANDATORY_CONFLICT", "conflicting_stops": [A, B], "required_hours": 10.75, "max_hours": 8.5, "reason": "Travel time between mandatory stops exceeds shift budget."}
    ```
    4. Require the manager to either extend shift authorization, convert some mandatory stops to flexible recommendations, or reassign stops to a peer.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/route_optimizer.py`, `pharmaHRMS_backend/app/routers/daily_plan.py`.
- **Dependencies:** OSRM travel matrix service.
- **Acceptance criteria:** Scheduling conflicting mandatory stops returns a validation error with actionable conflict details rather than an impossible route.

Issue 13: Midday Changes and Controlled Regeneration

- **Current implementation status:** Partially addressed.
- **What is missing:** Section 11.5 describes the replanning concept in prose, but there are no API endpoint specifications, no protection for stops currently in progress, no delay cascade logic, and no revision history tracking.
- **Example scenario:** Rajesh completes Stop 1 and Stop 2. While at Stop 3 (status = `ARRIVED`), Doctor 3 gets called into an emergency surgery, delaying the meeting by 60 minutes. Rajesh needs to replan his remaining afternoon stops. If he calls the existing `generate` endpoint, Stop 3 could be dropped or re-ordered while Rajesh is physically sitting in the clinic!
- **Technical impact:** Active visit sessions are corrupted, checked-in visits lose their session state, and downstream arrival times become inaccurate.
- **Recommended solution:**
    1. Add dedicated endpoint: `POST /api/v1/plans/{plan_id}/replan`.

2. Replan constraints:
    - **Locked Stops:** All stops with status `COMPLETED`, `SKIPPED`, `ARRIVED`, or `EN_ROUTE` are strictly frozen in their historical sequence.
    - **Dynamic Origin:** The origin coordinates for remaining stops are taken from the current active visit's location (or the representative's live GPS).
    - **Dynamic Start Time:** The start time is set to $\max(\text{current_time}, \text{active_visit_departure_time})$.
    - **Downstream Re-sequencing:** Only remaining `PENDING` stops are re-optimized via TSPTW. If the delay makes later stops infeasible, the lowest-priority non-mandatory stop is flagged for removal with representative confirmation.
3. Increment the plan version (`version = version + 1`, `revision_type = 'MIDDAY_REPLAN'`).

- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/routers/daily_plan.py`, `pharmaHRMS_backend/app/services/plan_optimizer.py`.
- **Dependencies:** Issue 6 revision schema.
- **Acceptance criteria:** Midday replanning recalculates only unvisited stops, preserves active and completed visits, and adjusts downstream ETAs to reflect actual meeting delays.

Issue 14: Route Service or Optimization Failure

- **Current implementation status:** Partially addressed.
- **What is missing:** The frontend client has an OSRM fallback (src/lib/osrm.ts: L66), but the backend optimization service does not exist. The backend architecture does not specify how matrix timeouts, disconnected graph nodes, or optimization exceptions are handled during batch generation.
- **Example scenario:** At 07:30 AM, the public OSRM server experiences a routing service outage or network partition. The backend optimization service attempts to fetch the 8×8 travel-time matrix. The HTTP request hangs and throws a 504 Gateway Timeout. The entire morning plan generation crashes for all 35 sales representatives.
- **Technical impact:** Total system failure at the start of the business day; sales representatives are blocked from planning visits and starting field duties.
- **Recommended solution:**
    1. Implement a resilient route-matrix client on the FastAPI backend:
        - Set a 4-second timeout on OSRM matrix requests.
        - If OSRM fails, automatically fall back to an internal Python Haversine matrix calculator using a $1.2\times$ road-winding factor and 30 km/h average urban speed.
        - Handle disconnected nodes: if OSRM returns `null` for specific matrix pairs (e.g. rural coordinates across unmapped terrain), substitute with Haversine distance for those specific cells.

2. Tag the generated plan with `routing_engine: 'OSRM_NETWORK' | 'HAVERSINE_ESTIMATED'`.
3. **Two-Phase In-Memory Commit:** Compute and validate the replacement route entirely in memory. Only commit changes to the database if the calculation succeeds. If replanning fails, preserve the existing active plan unchanged.

- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/route_optimizer.py`.
- **Dependencies:** None.
- **Acceptance criteria:** When OSRM is unreachable, backend plan generation completes successfully using the internal Haversine fallback, flags the itinerary as estimated, and never overwrites an existing valid plan with an error state.

Issue 15: Standalone Follow-ups Omitted

- **Current implementation status:** Partially addressed.
- **What is missing:** The architecture design strictly enforces `follow_ups.lead_id` as a non-null foreign key (Architecture Doc: L640). However, representatives also have general field duties and tasks in `pharma_hrms_tasks` (e.g., attending a medical conference, hospital stock audits, regulatory document drops) that do not originate from commercial leads. These are completely excluded from candidate ingestion.
- **Example scenario:** Manager Naidu assigns a critical task: *"Submit Form 12-B at State Drug Controller Office by 02:00 PM."* This task is in `pharma_hrms_tasks`, not linked to a lead. The AI plan generator only scans `leads` and their linked `follow_ups`. The Drug Controller visit is omitted from the route, causing Rajesh to miss the statutory submission deadline.
- **Technical impact:** Non-lead field obligations are ignored by automated planning, forcing representatives to maintain a parallel manual paper or calendar schedule.
- **Recommended solution:**
  1. Support both **Lead Obligations** and **Task Obligations** in candidate loading:

```
# Ingestion Pipeline:
lead_obligations = load_lead_candidates(employee_id)
task_obligations = load_uncompleted_tasks(employee_id, target_dat
```

  2. For tasks in `pharma_hrms_tasks` that require physical presence and have assigned coordinates, convert them into candidate stops with `stop_type = 'MANAGER_MANDATORY'` or `'TASK_VISIT'`.
  3. For tasks without coordinates, treat them as administrative time blocks (see Issue 16).
  4. Preserve Critical Product Constraint #1: Do **not** create artificial lead records for administrative tasks. The junction table `daily_sales_plan_item_obligations` links directly to `pharma_hrms_tasks(id)` when `source_type = 'TASK'`.

- **Files/tables/APIs affected:** `pharmaHRMS_backend/app/services/plan_optimizer.py`, table `daily_sales_plan_item_obligations`.
- **Dependencies:** Issue 2 junction table.
- **Acceptance criteria:** Physical tasks assigned in `pharma_hrms_tasks` are ingested as valid candidate stops alongside lead visits, without generating artificial rows in the `leads` table.

Issue 16: Non-Visit Actions Handled as Route Stops

- **Current implementation status:** Missing.
- **What is missing:** The current architecture models all planned items as geographic route stops with latitude, longitude, and OSRM polyline routing. It provides no mechanism for non-visit actions (phone follow-ups, quotation drafting, clinical document submissions) to exist in the daily plan without distorting the geographic route optimizer.
- **Example scenario:** Rajesh has 5 customer phone calls to make and a quotation to draft for Apollo Hospital. Because these are marked as follow-ups in the system, the optimizer treats them as physical stops. Because they lack coordinates, the optimizer either crashes or places them at $(0, 0)$. If they are omitted, the system schedules 8.5 hours of driving and visits, leaving Rajesh zero time to make his phone calls.
- **Technical impact:** Geographic optimizer crashes on missing coordinates, or generates an unachievable schedule that ignores essential administrative and phone call duties.
- **Recommended solution:**
    1. Add `activity_type` to `daily_sales_plan_items`:
        - `PHYSICAL_VISIT`: Requires travel and GPS check-in (sent to OSRM and 2-Opt TSPTW).
        - `PHONE_CALL`: Remote customer interaction (scheduled during office hours or transit pauses).
        - `ADMIN_DESK`: Quotation preparation, expense filing, stockist reconciliations.
    2. **Dedicated Time Budgeting:** Remote activities are given fixed schedule blocks (e.g., 08:30–09:30 AM for morning calls, 05:00–06:00 PM for evening reporting).
    3. The route optimizer subtracts non-visit blocks from the total available field shift duration before optimizing the physical travel path:
    $$\text{Field Travel Budget} = \text{Shift Budget} - \text{Lunch} - \sum \text{Duration}(\text{Non-Visit Activities})$$
- **Files/tables/APIs affected:** Table `daily_sales_plan_items`, `pharmaHRMS_backend/app/services/plan_optimizer.py`.
- **Dependencies:** Plan item schema.
- **Acceptance criteria:** Phone calls and desk tasks are scheduled in the daily itinerary with appropriate time allocations, but are excluded from the OSRM road matrix and geographic route

map.

Issue 17: Plan Status Confused with Execution Status

- **Current implementation status:** Partially addressed.
- **What is missing:** In the current design and in `field-visit-service.ts` (src/lib/services/field-visit-service.ts: L249), tapping "Check Out" marks the visit as `COMPLETED`. The architecture assumes that visit completion automatically marks all linked follow-ups and tasks complete. It lacks structured outcome capturing and fails to separate physical execution from business milestone achievement.
- **Example scenario:** Rajesh arrives at Seven Hills Clinic for a high-priority follow-up to obtain a signed purchasing contract. He checks in via GPS. However, the doctor informs him that the hospital board delayed the budget until next month. Rajesh checks out. If the system blindly marks the follow-up as `COMPLETED` and advances the lead to `WON`, the business records are completely falsified.
- **Technical impact:** Inaccurate pipeline stage reporting, premature closure of unfulfilled customer commitments, and falsification of sales revenue metrics.
- **Recommended solution:**
  1. **Strict Decoupling of States:**
     - **Visit Execution Status** (`pharma_hrms_field_visits.status`): Records physical presence (`ASSIGNED` $\to$ `EN_ROUTE` $\to$ `ARRIVED` $\to$ `COMPLETED`).
     - **Plan Item Status** (`daily_sales_plan_items.status`): Tracks schedule progression (`PENDING` $\to$ `IN_PROGRESS` $\to$ `VISITED` $\to$ `SKIPPED`).
     - **Obligation Outcome Status** (`daily_sales_plan_item_obligations.fulfillment_status`): Records business results (`FULFILLED`, `UNFULFILLED_RESCHEDULED`, `CANCELLED`).
  2. **Mandatory Checkout Outcome Modal:** On checkout, the representative must specify:
     - Visit outcome: `SUCCESSFUL_MEETING`, `DOCTOR_UNAVAILABLE`, `RESCHEDULE_REQUESTED`, `GATEKEEPER_BLOCKED`.
     - For each linked obligation: toggle whether the commitment was fulfilled or needs a new follow-up date.
     - Updated lead stage (with validation rules).
- **Files/tables/APIs affected:** `Pharma_HRMS/src/components/CheckInModule.tsx`, `Pharma_HRMS/src/lib/services/field-visit-service.ts`, `pharmaHRMS_backend/app/routers/field.py`.
- **Dependencies:** Issue 2 junction table.
- **Acceptance criteria:** A completed GPS visit record can close with an outcome of "Doctor Unavailable", keeping the underlying follow-up open and prompting for a reschedule date without

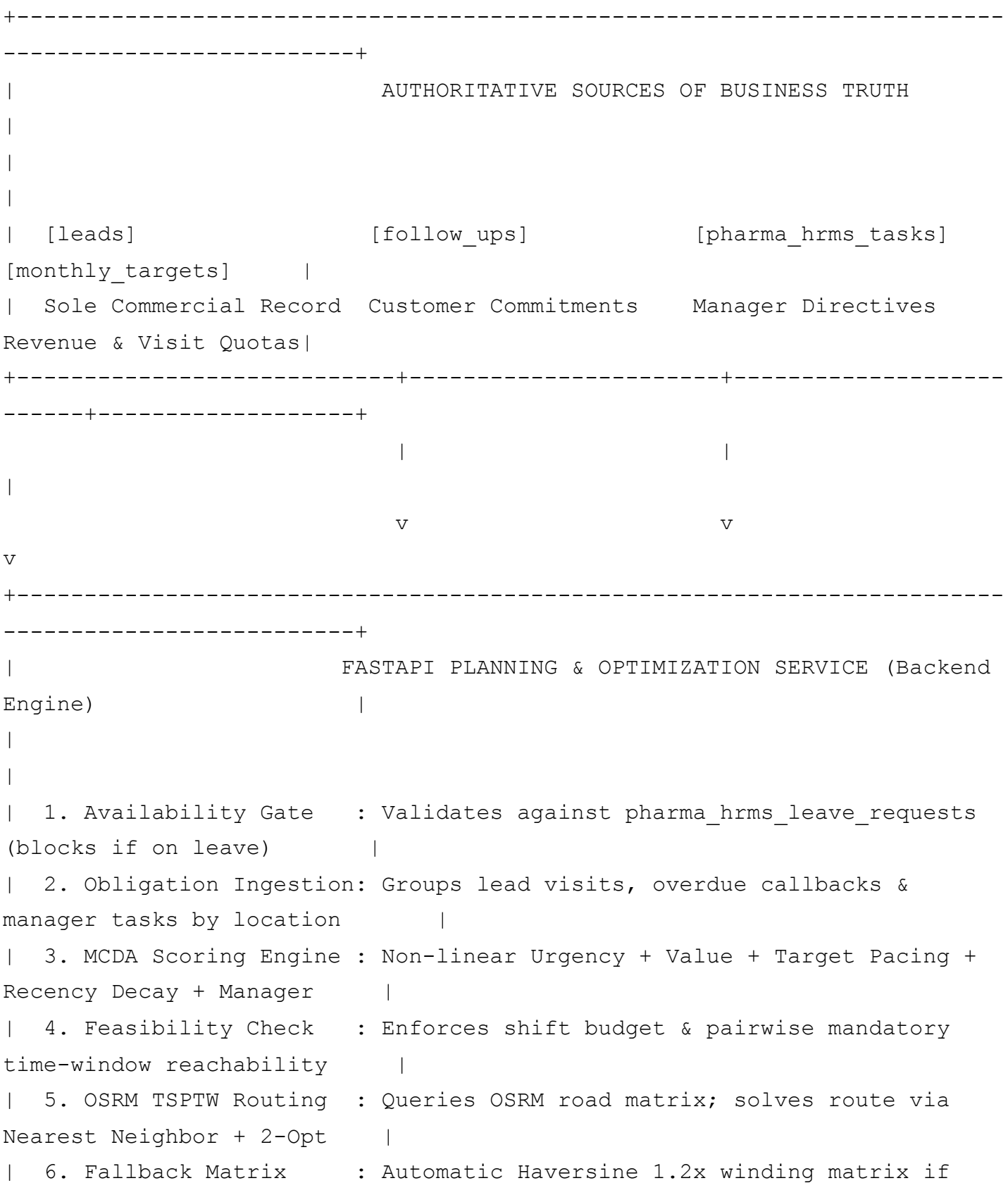
falsifying deal stage.

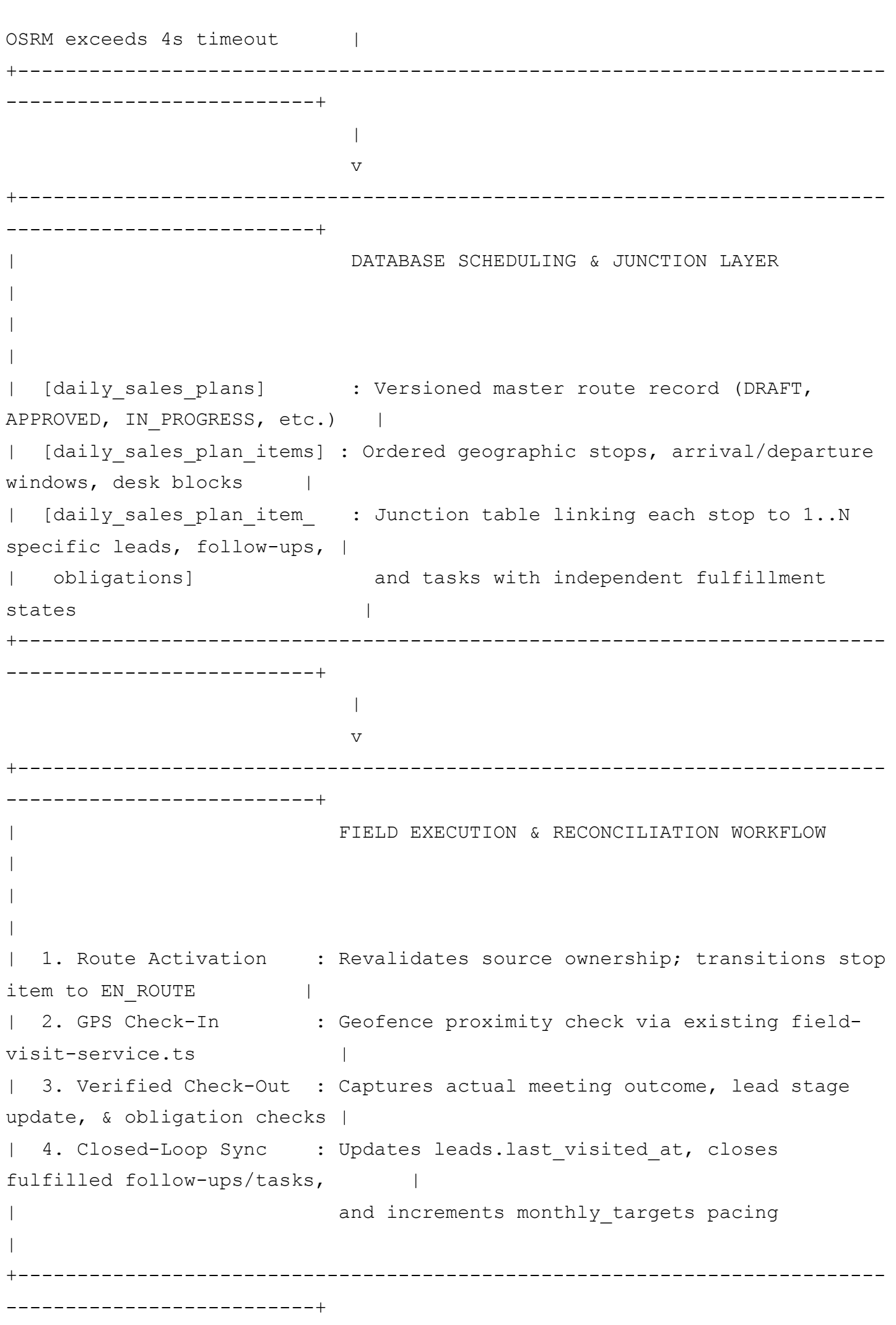
Issue 18: Authorization and Source Ownership Drift

- **Current implementation status:** Missing.
- **What is missing:** Database migrations define open RLS policies (`CREATE POLICY "%s_all" ON "%I" FOR ALL USING (true);` in 00_pharma_hrms_full_migration.sql: L457). In the frontend, the Supabase client connects with an anonymous key (src/core/supabase.ts: L18) without Supabase Auth session tokens (`persistSession: false`). The design assumption that PostgreSQL RLS inspects `auth.uid()` is completely invalid.
- **Example scenario:** Representative Naveen is terminated or transferred to Hyderabad. His mobile client still holds an old cached session in `localStorage` . If the client makes direct PostgREST calls to Supabase, all table records across all territories can be read and updated because the active RLS policy is `USING (true)` and the anon key is public in the frontend bundle.
- **Technical impact:** Critical security vulnerability: complete bypass of organizational hierarchy access controls, cross-territory data exposure, and unauthorized record tampering.
- **Recommended solution:**
  1. **Enforce Backend Mediation:** All read and write operations for the Daily Sales Plan, Leads, and Field Visits must be routed through FastAPI backend endpoints, which enforce JWT token verification via app/dependencies/auth.py.
  2. **Server-Side Hierarchy Authorization:**
     - Business Executives (BE): Access restricted strictly to records where `assigned_to = current_user.id` .
     - Regional Sales Managers (RSM): Access restricted to employees reporting directly to them (`pharma_hrms_employees.reporting_to = current_user.id`).
     - Zonal Sales Managers (ZSM): Access restricted to their geographic zone (`pharma_hrms_employees.zone = current_user.zone`).
     - Admins: System-wide access.
  3. **Lock Down Database RLS:** Remove the permissive `USING (true)` policies. For production hardening, configure Supabase with custom JWT verification matching the FastAPI signing secret, or revoke PostgREST public schema access so all traffic is channeled through FastAPI.
- **Files/tables/APIs affected:** `pharmaHRMS_backend/database/00_pharma_hrms_full_migration.sql` , `pharmaHRMS_backend/app/dependencies/auth.py` , `pharmaHRMS_backend/app/routers/daily_plan.py` .
- **Dependencies:** FastAPI auth infrastructure.
- **Acceptance criteria:** A representative attempting to read or modify a sales plan or lead belonging to another representative receives HTTP 403 Forbidden, enforced strictly on the server.

5. Corrected Architecture and End-to-End Data Flow

The corrected architecture positions the **AI Daily Sales Plan** strictly as an intelligent recommendation and scheduling layer. It does not own commercial truth; it aggregates obligations from authoritative business records, organizes them into a feasible geographic and chronological itinerary, and reconciles execution results back to the authoritative modules.

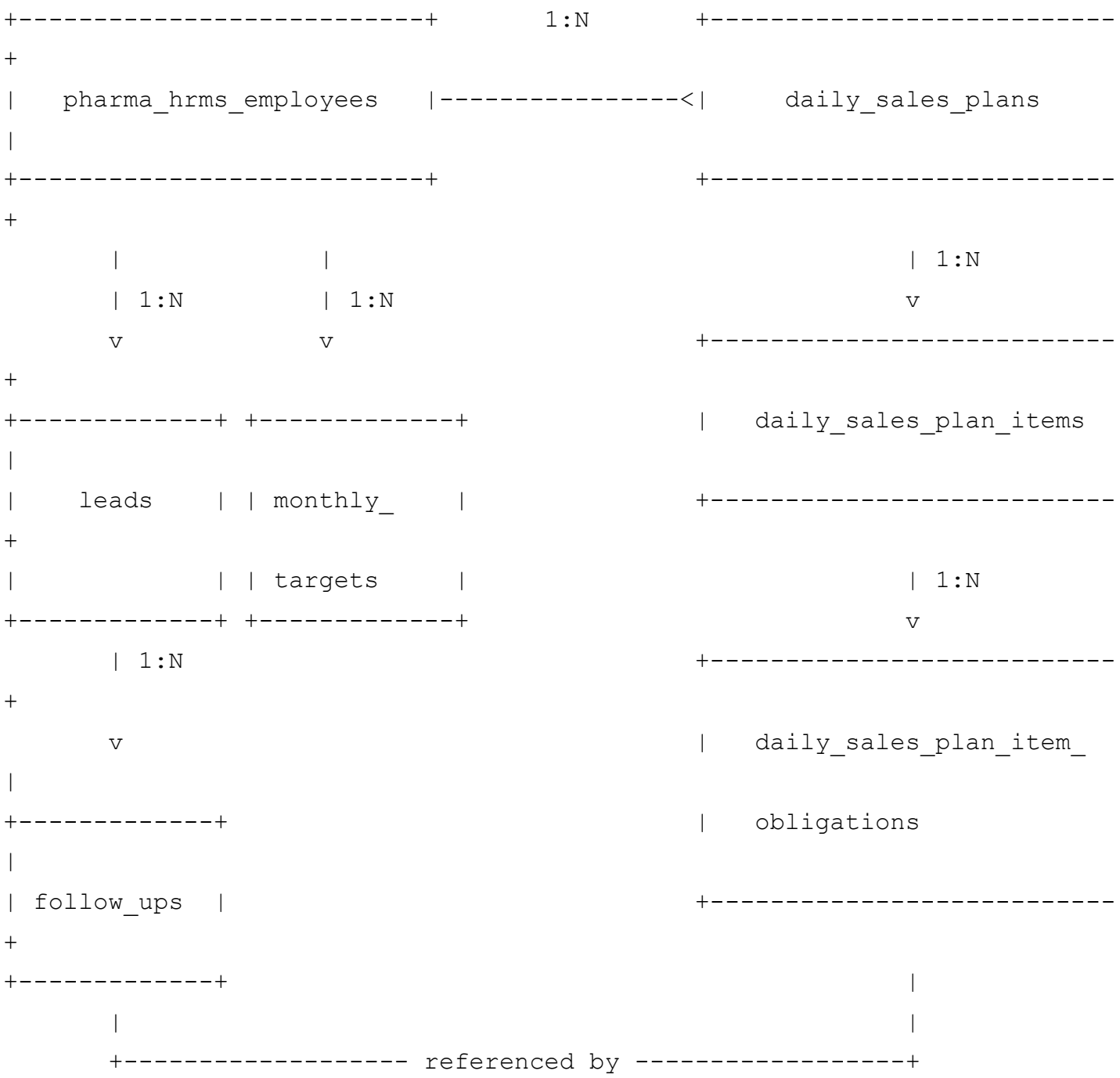



OSRM exceeds 4s timeout
DATABASE SCHEDULING & JUNCTION LAYER
[daily_sales_plans] : Versioned master route record (DRAFT, APPROVED, IN_PROGRESS, etc.)
[daily_sales_plan_items] : Ordered geographic stops, arrival/departure windows, desk blocks
[daily_sales_plan_item_ obligations] : Junction table linking each stop to 1..N specific leads, follow-ups, and tasks with independent fulfillment states
FIELD EXECUTION & RECONCILIATION WORKFLOW
1. Route Activation : Revalidates source ownership; transitions stop item to EN_ROUTE
2. GPS Check-In : Geofence proximity check via existing field-visit-service.ts
3. Verified Check-Out : Captures actual meeting outcome, lead stage update, & obligation checks
4. Closed-Loop Sync : Updates leads.last_visited_at, closes fulfilled follow-ups/tasks, and increments monthly_targets pacing

6. Proposed Database Relationship Changes

To support multi-obligation stops, historical plan versioning, and referential integrity without duplicating commercial records, the following relational data model is proposed:

6.1 Entity Relationship Diagram



6.2 Data Definition Language (DDL) Specifications

1. Master Table: `daily_sales_plans`

```
CREATE TABLE IF NOT EXISTS daily_sales_plans (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    employee_id VARCHAR(255) NOT NULL REFERENCES pharma_hrms_employees(id
    plan_date DATE NOT NULL,
    version INTEGER NOT NULL DEFAULT 1,
    revision_type VARCHAR(50) NOT NULL DEFAULT 'AI_INITIAL',
    status VARCHAR(50) NOT NULL DEFAULT 'DRAFT',
    total_planned_stops INTEGER NOT NULL DEFAULT 0,
    total_planned_distance_meters NUMERIC(10, 2) DEFAULT 0.00,
    total_planned_duration_seconds INTEGER DEFAULT 0,
    routing_engine VARCHAR(50) DEFAULT 'OSRM_NETWORK',
    approved_by VARCHAR(255) REFERENCES pharma_hrms_employees(id),
    approved_at TIMESTAMP WITH TIME ZONE,
    idempotency_key UUID UNIQUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_plan_emp_date_version UNIQUE (employee_id, plan_date, v
);
```

2. Item Table: `daily_sales_plan_items`

```
CREATE TABLE IF NOT EXISTS daily_sales_plan_items (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    plan_id UUID NOT NULL REFERENCES daily_sales_plans(id) ON DELETE CASC
    sequence_order INTEGER NOT NULL,
    activity_type VARCHAR(50) NOT NULL DEFAULT 'PHYSICAL_VISIT',
    planned_arrival_time TIME,
    planned_departure_time TIME,
    estimated_duration_minutes INTEGER NOT NULL DEFAULT 30,
    lead_id UUID REFERENCES leads(id) ON DELETE SET NULL,
    assigned_latitude NUMERIC(10, 7),
    assigned_longitude NUMERIC(10, 7),
    assigned_address TEXT,
    stop_type VARCHAR(50) NOT NULL DEFAULT 'AI_RECOMMENDED',
    recommendation_score NUMERIC(5, 2),
    recommendation_reasons JSONB DEFAULT '[]'::jsonb,
    status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
    field_visit_id UUID REFERENCES pharma_hrms_field_visits(id) ON DELETE
    transit_distance_from_prev_m NUMERIC(10, 2) DEFAULT 0.00,
    transit_duration_from_prev_s INTEGER DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT uq_item_plan_sequence UNIQUE (plan_id, sequence_order)
);
```

3. Junction Table: `daily_sales_plan_item_obligations`

```
CREATE TABLE IF NOT EXISTS daily_sales_plan_item_obligations (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    plan_item_id UUID NOT NULL REFERENCES daily_sales_plan_items(id) ON D
    source_type VARCHAR(50) NOT NULL, -- 'LEAD_CONVERSION', 'FOLLOW_UP',
    source_id UUID NOT NULL,
    obligation_title TEXT NOT NULL,
    is_mandatory BOOLEAN NOT NULL DEFAULT FALSE,
    fulfillment_status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
    outcome_notes TEXT,
    fulfilled_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

4. Historical Revision Snapshots: `daily_sales_plan_revisions`

```
CREATE TABLE IF NOT EXISTS daily_sales_plan_revisions (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    plan_id UUID NOT NULL REFERENCES daily_sales_plans(id) ON DELETE CASC
    version INTEGER NOT NULL,
    revision_type VARCHAR(50) NOT NULL,
    plan_snapshot JSONB NOT NULL,
    created_by VARCHAR(255) REFERENCES pharma_hrms_employees(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

---

7. API and Authorization Changes

All Daily Sales Plan operations must be routed through FastAPI to enforce server-side organizational hierarchy authorization.

API Specifications

1. Generate Daily Plan

- **Method & Path:** `POST /api/v1/plans/generate`
- **Access:** Sales Representative (for self), RSM / Admin (for direct subordinates).
- **Request Payload:**

```
{
  "employee_id": "OL013",
  "plan_date": "2026-10-22",
  "start_latitude": 17.7288,
  "start_longitude": 83.3154,
  "max_stops": 6,
  "idempotency_key": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "force_regenerate": false
}
```

- **Status Codes:**
  - `201 Created` : Plan successfully generated.
  - `409 Conflict` : Active approved/in-progress plan already exists.
  - `422 Unprocessable Entity` : Employee on approved leave, or mandatory stop feasibility conflict.

2. Get Daily Plan with Real-Time Reconciliation

- **Method & Path:** `GET /api/v1/plans/daily?employee_id=OL013&date=2026-10-22`
- **Access:** BE (self), RSM (reporting team), ZSM (zone), Admin (all).
- **Processing:** Reconciles unexecuted items against authoritative source states before returning the payload.
- **Status Codes:** `200 OK`, `403 Forbidden`, `404 Not Found`.

3. Midday Replanning

- **Method & Path:** `POST /api/v1/plans/{plan_id}/replan`
- **Access:** Assigned BE or reporting RSM.
- **Request Payload:**

```
{
  "current_latitude": 17.7312,
  "current_longitude": 83.3190,
  "replan_reason": "MEETING_OVERRUN_AT_STOP_2"
}
```

- **Processing:** Freezes completed/arrived stops; re-optimizes remaining pending stops from current location; increments plan version.

- **Status Codes:** `200 OK`, `400 Bad Request` (no pending stops left).

4. Revalidate and Start Stop Execution

- **Method & Path:** `POST /api/v1/plans/items/{item_id}/start`
- **Access:** Assigned representative only.
- **Processing:** Re-validates that the lead is still assigned to `current_user.id` and that stage is active. Transitions item to `EN_ROUTE` and creates/links `pharma_hrms_field_visits` row.
- **Status Codes:** `200 OK`, `409 Conflict` (lead reassigned or closed).

5. Record Visit Outcome & Obligation Fulfillment

- **Method & Path:** `POST /api/v1/plans/items/{item_id}/complete`
- **Access:** Assigned representative only.
- **Request Payload:**

```
{
  "actual_latitude": 17.7289,
  "actual_longitude": 83.3156,
  "arrival_distance_m": 24.5,
  "duration_minutes": 32,
  "visit_outcome": "PROPOSAL_ACCEPTED",
  "updated_lead_stage": "NEGOTIATION",
  "obligations": [
    {
      "obligation_id": "b1c2d3e4-0000-1111-2222-333344445555",
      "fulfilled": true,
      "notes": "Delivered revised cardiac trial documentation."
    }
  ],
  "next_follow_up": {
    "scheduled_date": "2026-10-27",
    "scheduled_time": "14:00:00",
    "title": "Final Contract Signing",
    "urgency": "HIGH"
  }
}
```

- **Processing:** Atomically updates visit record, closes fulfilled obligations, updates lead stage and `last_visited_at`, and schedules next follow-up.

---

8. Scheduling and Route-Feasibility Rules

To prevent false ETAs, unrealistic travel expectations, and impossible schedules, the optimization engine must enforce the following deterministic rules:

1. Activity Classification & Time Budgeting

Every daily schedule operates on an 8.5-hour standard workday budget (510 minutes):
$$\text{Available Field Transit & Visit Time} = 510 - 45\text{ (Lunch)} - \sum \text{Duration} (\text{Desk/Remote Activities})$$

2. Configurable Visit Durations by Category

Visit Category	Default Duration	Minimum Geofence Radius	Scheduling Type
Routine Doctor Detailing	20 minutes	300 meters	Geographic Stop
KOL / Specialist Consultation	35 minutes	300 meters	Geographic Stop
Hospital Purchase Committee	60 minutes	500 meters	Geographic Stop (Fixed Window)
Pharmacy / Retail Stock Check	15 minutes	150 meters	Geographic Stop
Customer Phone Callback	10 minutes	N/A (Remote)	Non-Visit Block
Quotation / Admin Desk Work	45 minutes	N/A (Remote)	Non-Visit Block
Mandatory Lunch Break	45 minutes	N/A	Midday Window (13:00–14:15)

3. Non-Linear MCDA Scoring Formula

Priority scores ($S_i \in [0, 100]$) are computed deterministically:
$$S_i = \min\left(100, ; 0.30 \cdot U_i + 0.25 \cdot V_i + 0.20 \cdot T_i + 0.15 \cdot D_i + 0.10 \cdot M_i\right)$$

- **Urgency (U_i):**

    - Confirmed appointment due today: 100
    - Overdue 1–3 days: 85
    - Due today (flexible time): 70
    - Due within 48 hours: 40
    - Overdue >7 days (chronic): 30 (triggers leakage alert; does not monopolize route)
    - No active follow-up: 10
- **Commercial Value (V_i):** Normalized against territory max deal value:
  $$V_i = \min\left(100, ; \frac{\text{deal_value}_i}{\text{MaxTerritoryDealValue}} \times 100\right)$$
- **Target Pacing Deficit (T_i):** Scales when representative is behind quota:
  $$T_i = \min\left(100, ; \frac{\text{Revenue Target} - \text{Revenue Achieved}}{\text{Revenue Target}} \times \frac{\text{deal_value}_i}{\text{AvgDealValue}} \times 50\right)$$
- **Recency Decay (D_i):** Prevents account neglect:
  $$D_i = \min\left(100, ; \frac{\text{Days Since Last Visit}}{30} \times 100\right)$$
- **Manager Priority (M_i):** Task directive weight (`URGENT` $= 100$, `HIGH` $= 75$, `NORMAL` $= 40$, `NONE` $= 10$).

4. Mandatory Feasibility & Window Overlap Check

Before executing 2-Opt route sequencing, candidate stops must pass reachability verification:

1. For any two stops A and B with fixed appointment windows $[\text{start}_A, \text{end}_A]$ and $[\text{start}_B, \text{end}_B]$:
   $$\text{start}_B \ge \text{start}_A + \text{duration}A + T{\text{road}}(A, B)$$
2. If total travel time + total meeting durations exceed 8.5 hours, and mandatory stops alone exceed 8.5 hours, the solver aborts with `MANDATORY_CONFLICT` and alerts the manager.

---

9. Testing and Acceptance Criteria

Unit Test Suites

- **Scoring Normalization (`test_lead_scorer.py`):**
    - Verify that a confirmed appointment due today scores higher than a 14-day overdue low-value follow-up.
    - Verify zero-division protection when revenue achieved $\ge$ revenue target.
    - Verify missing deal value or unconfigured monthly target falls back gracefully to neutral defaults.
- **Route Heuristics (`test_route_optimizer.py`):**
    - Verify 2-Opt removes crisscrossing loops on a known 6-point coordinate graph.

    - Verify lunch break is inserted strictly between 13:00 and 14:15.
    - Verify mandatory stops are not dropped during shift budget truncation.
- **Conflict Detector (`test_feasibility.py`):**
    - Test overlapping mandatory appointment windows: assert HTTP 422 returned with conflicting IDs.

Integration & Security Test Suites

- **Idempotency & Concurrency (`test_plan_concurrency.py`):**
    - Simulate 5 parallel POST requests with the same employee ID and date: verify exactly one plan master is created.
- **Authorization & Drift (`test_plan_auth.py`):**
    - BE user attempts to generate plan for peer BE: assert HTTP 403 Forbidden.
    - Reassign lead after plan creation: assert subsequent start call returns HTTP 409 Conflict.
- **Midday Replanning (`test_replan.py`):**
    - Complete 2 stops, set stop 3 to `ARRIVED` , call replan: verify stops 1, 2, 3 retain their order and sequence numbers while stops 4, 5, 6 are re-sequenced from stop 3 coordinates.

---

10. Prioritized Implementation Plan

Group 1: Required Before Implementation Approval (Architectural Prerequisites)

1. Approval of logical schema and `daily_sales_plan_item_obligations` junction design.
2. Sign-off on the revised non-linear MCDA formula and chronic backlog cap.
3. Sign-off on mandatory stop conflict rejection policy (HTTP 422 vs. auto-rescheduling).

Group 2: Required for First Production Release (Phase 1–4)

1. Database migrations creating `leads` , `follow_ups` , `monthly_targets` , `daily_sales_plans` , `daily_sales_plan_items` , and `daily_sales_plan_item_obligations` .
2. SQLAlchemy models in `pharmaHRMS_backend/app/models/plan.py` and `lead.py` .
3. Multi-factor lead scorer and OSRM + Haversine fallback matrix client in FastAPI.
4. Nearest Neighbor + 2-Opt TSPTW solver with category visit durations and lunch break.
5. FastAPI plan router (`/generate` , `/daily` , `/replan` , `/complete`) with server-side authorization.

6. Refactoring `DoctorVisitPlanner.tsx` to display AI recommendations, why-badges, Leaflet route preview, and checkout outcome modal.

Group 3: Later Improvements (Post-V1 Optimization)

1. Live vehicle telemetry ingestion for dynamic traffic updates.
2. Self-hosted containerized OSRM service deployed via Docker.
3. Multi-day territory planning and overnight stay routing for rural zones.

---

11. Outstanding Business Decisions Requiring Approval

The following four business policies must be resolved by product leadership before starting Phase 1:

1. **Mandatory Stop Infeasibility Resolution:**
   - *Option A (Recommended):* Strict rejection. If mandatory stops cannot fit within 8.5 hours or violate time windows, return HTTP 422 with conflict details and require manager adjustment.
   - *Option B:* Permissive overflow. Automatically extend shift hours and mark overtime on the schedule.
2. **Manager Approval Timeout Policy:**
   - *Option A (Recommended):* Auto-approve at 09:00 AM. If a manager does not review a representative's plan by 09:00 AM, the plan transitions to `APPROVED` to avoid stalling field operations.
   - *Option B:* Strict approval. Representatives cannot start visits until explicit manager sign-off is logged.
3. **Chronic Backlog SLA Threshold:**
   - Confirm whether 7 days or 14 days overdue constitutes "chronic backlog" to be capped at $U_i = 40$ and diverted to Revenue Leakage alerts.
4. **Lead Stage Transition Permissions:**
   - Confirm whether sales representatives can transition leads directly to `WON` or `LOST`, or if closing a deal requires manager verification.

---

12. Final Recommendation on Whether Architecture is Ready

Recommendation: **HOLD PENDING REVISED SCHEMA AND POLICY APPROVAL**

The initial architecture was a strong conceptual overview but lacked the relational rigidity, concurrency safety, and execution reconciliation required for production field sales operations.

By incorporating the corrections defined in this report:

- The junction model resolves multi-obligation stops (Issues 2, 3, 15).
- Server-side FastAPI authorization and lockouts resolve security drift (Issues 4, 9, 18).
- Versioned revisions and non-cloning rollover resolve data integrity (Issues 1, 6, 7).
- Non-linear scoring, category durations, and conflict detection resolve route feasibility (Issues 8, 11, 12, 14, 16).
- Decoupled execution and outcome-verified checkout resolve business truth (Issues 5, 10, 13, 17).

Once leadership signs off on the four business decisions in Section 11, the engineering team can proceed directly to Phase 1 database migrations.